بشکنند. اگر پردازه‌های محاوره‌ای نسبت به پردازه‌های غیرمحاوره‌ای الویت بگیرند، در این صورت ممکن است که کاربران سراغ پردازه‌های محاوره‌ای بروند. این مسئله معمولاً با استفاده از ابزار یا اسکریپت‌هایی که مجموعه‌های کاملی از عملیات را به‌صورت کپسول دارند، با استفاده‌ی مکرر از ابزار، و استفاده از ابزار در حالی که نتایج را اندازه‌گیری می‌کنند (و تشخیص هر مسئله‌ای که در محیط باعث می‌شوند).

البته، رفتار انسانی یا برنامه می‌تواند الگوریتم‌های زمان‌بندی را با چالش مواجه کند. برای مثال، محققان سیستمی طراحی کردند که با نگاه خودکار به حجم I/O در ترمینال، پردازه‌ها را به محاوره‌ای و غیرمحاوره‌ای دسته‌بندی می‌کرد. اگر پردازه‌ای در فاصله‌ی ۱ ثانیه‌ای، ورودی یا خروجی به ترمینال نداشت، در رده‌ی پردازه‌های غیرمحاوره‌ای قرار می‌گرفت و به یک صف با الویت پایین‌تر جابجا می‌شد. یک برنامه‌نویس برای مقابله با این سیاست، برنامه‌هایش را طوری تغییر داد که کاراکتر دلخواهی در فواصل منظم کمتر از ۱ ثانیه در ترمینال بنویسد. سیستم به برنامه‌های این برنامه‌نویس الویت بالا داد، هرچند که خروجی ترمینال کاملاً بی‌معنی بود.

در حالت کلی، انعطاف‌پذیرترین الگوریتم‌های زمانبندی آنهایی هستند که توسط مدیران سیستم یا کاربران قابل تغییر هستند، طوری که با یک برنامه یا مجموعه‌ای از برنامه‌های کاربردی خاص تطبیق یابند. برای نمونه، نیازهای زمانبندی یک ایستگاه‌کاری که با برنامه‌های گرافیکی کاربردی سطح بالا سر و کار دارد، در مقایسه با یک کارگزار وب یا یک کارگزار فایل متفاوت است. برخی سیستم‌عامل‌ها—به‌خصوص نسخه‌های متعددی از یونیکس—به مدیر سیستم این اجازه را می‌دهند که پارامترهای زمانبندی را مناسب پیکربندی یک سیستم خاص تنظیم نماید. برای مثال، سولاریس فرمان dispadmin را فراهم می‌کند تا مدیر سیستم بتواند پارامترهای رده‌های زمانبندی مطرح شده در بخش ۵-۷-۳ را تغییر بدهد.

رویکرد دیگر، استفاده از APIهایی است که الویت یک پردازه یا یک نخ را تغییر می‌دهند. APIهای جاوا، پوزیکس و ویندوز چنین توابعی را فراهم می‌کنند. نقطه ضعف این روش آن است که تغییر آهنگ کارایی یک سیستم یا یک برنامه‌ی کاربردی اکثراً موجب بهبود کارایی در شرایط کلی‌تر نمی‌شود.

۵-۹ خلاصه

- زمانبندی پردازنده، کار انتخاب پردازه‌ای منتظر از صف آماده و تخصیص پردازنده به آن می‌باشد. پردازنده توسط اعزام‌کننده در اختیار پردازه‌ی انتخابی قرار می‌گیرد.
- الگوریتم‌های زمان‌بندی می‌توانند پس‌گرفتنی (جایی که پردازنده می‌تواند از پردازه گرفته شود) و یا غیرقابل پس‌گیر باشند (جایی که یک پردازه بایستی داوطلبانه کنترل پردازنده را رها کند). تقریباً تمای سیستم‌عامل‌های مدرن پس‌گرفتنی هستند.
- الگوریتم‌های زمان‌بندی می‌توانند مطابق پنج معیار ارزیابی شوند که عبارتند از: (۱) بهره‌وری پردازنده، (۲) بازده، (۳) زمان گردش، (۴) زمان انتظار، و (۵) زمان پاسخ.

- زمانبندی اولین-ورود، اول سرویس (FCFS)، ساده‌ترین الگوریتم زمانبندی است، اما می‌تواند باعث شود که پردازه‌های کوتاه به انتظار پردازه‌های بسیار بزرگ بمانند.
- زمانبندی کوتاه‌ترین-کار-اول (SJF) بهینه بوده و کوتاه‌ترین میانگین زمان انتظار را دارد. با این وجود، چون پیش‌بینی طول انفجار بعدی پردازنده مشکل است، پیاده‌سازی زمانبندی SJF نیز مشکل خواهد بود.
- زمان‌بندی نوبت-چرخشی (RR) پردازنده را برای یک کمیت زمانی در اختیار هر پردازه می‌گذارد. اگر پردازه پردازنده را پیش از این که کمیت زمانیش سپری شود رها نکند، پردازه پس‌گرفته شده و پردازه‌ی دیگری برای اجرای کمیت زمانیش زمان‌بندی می‌شود.
- زمان‌بندی الویتی برای هر پردازه یک الویت نسبت می‌دهد، و پردازنده به پردازه‌ای اختصاص داده می‌شود که بالاترین الویت را دارد. پردازه‌های با الویت یکسان می‌توانند به‌ترتیب FCFS یا با استفاده از زمان‌بندی RR زمان‌بندی شوند.
- زمان‌بندی صف چندسطحی، پردازه‌ها را به چندین صف مجزا با ترتیب الویتی تفکیک کرده و زمان‌بند، پردازه‌های صف با بالاترین الویت را اجرا می‌کند. الگوریتم‌های زمان‌بندی متفاوتی ممکن است در هر یک از صف‌ها استفاده شود.
- صف‌های با بازخورد چندسطحی مشابه صف‌های چندسطحی هستند، به‌استثنای این که یک پردازه می‌تواند در میان صف‌های متفاوت مهاجرت داشته باشد.
- پردازنده‌های چندهسته‌ای، یک یا چند CPU را بر روی تراشه‌ی فیزیکی یکسانی قرار داده و هر CPU می‌تواند بیشتر از یک نخ سخت‌افزاری داشته باشد. از نقطه‌نظر سیستم‌عامل، هر نخ سخت‌افزاری مثل یک CPU منطقی است.
- تعادل بار بر روی سیستم‌های چندهسته‌ای بارها را میان هسته‌های CPU برابر می‌سازد، هرچند که مهاجرت نخ‌ها میان هسته‌ها به‌منظور موازنه‌ی بارها ممکن است محتویات کَش را بی‌اعتبار ساخته و لذا ممکن است زمان‌های دسترسی حافظه را افزایش بدهد.
- زمان‌بندی بلادرنگ نرم، الویت را به وظایف بلادرنگ می‌دهد تا وظایف غیربلادرنگ. زمان‌بندی بلادرنگ سخت برای وظایف بلادرنگ، تضمین زمان اجرا فراهم می‌کند.
- زمان‌بندی نرخ-یکنواخت، وظایف متناوب را با استفاده از یک سیاست الویت ایستا با پس‌گیری زمان‌بندی می‌کند.
- زمان‌بندی قدیمی‌ترین-ضرب‌الاجل-اول (EDF)، الویت‌ها را مطابق ضرب‌الاجل نسبت می‌دهد. هرچه ضرب‌الاجل قدیمی‌تر باشد، الویت بالاتر است؛ هرچه ضرب‌الاجل بعدتر باشد، الویت پایین‌تر است.
- زمان‌بندی سهم متناسب، T سهم را مابین تمامی برنامه‌ها تقسیم می‌کند. اگر به

برنامه‌ای N سهم زمان اختصاص یابد، تضمین است که N/T زمان کل پردازنده را در اختیار خواهد داشت.

- لینوکس از زمان‌بندی کاملاً منصفانه (CFS) استفاده می‌کند که بخشی از زمان پردازش CPU را به هر وظیفه نسبت می‌دهد. نسبت، بر اساس مقدار زمان اجرای مجازی (vruntime) مرتبط با هر وظیفه است.
- زمان‌بندی ویندوز از یک طرح الویت ۳۲ سطحی پس‌گیر برای تعیین ترتیب زمان‌بندی نخ استفاده می‌کند.
- سولاریس، شش رده‌ی زمان‌بندی یکتا را مشخص می‌کند که به یک الویت سراسری نگاشته می‌شوند. نخ‌های پاپیچ پردازنده عموماً به الویت‌های پایین‌تر گمارده (و کمیت‌های زمانی طولانی‌تر) می‌شوند، و نخ‌های پاپیچ I/O معمولاً به‌الویت‌های بالاتر (با کمیت‌های زمانی کوتاه‌تر) گمارده می‌شوند.
- مدل‌سازی و شبیه‌سازی می‌تواند برای ارزیابی یک الگوریتم زمان‌بندی پردازنده استفاده شود.

خودآزمایی

۵-۱ الگوریتم زمانبندی پردازنده، یک ترتیب برای اجرای پردازه‌های زمانبندی شده‌اش معین می‌کند. اگر n پردازه برای زمانبندی روی یک پردازنده داشته باشیم، چه تعداد زمانبندی متفاوت امکان‌پذیرند؟ فرمولی برحسب n ارائه دهید.

۵-۲ تفاوت میان زمانبندی پس‌گرفتنی و غیرقابل پس‌گیر را توضیح دهید.

۵-۳ فرض کنید که پردازه‌های زیر در زمان‌های داده شده برای اجرا می‌رسند. هر یک از پردازه‌ها به مدت زمانی که لیست شده است اجرا خواهند داشت. در پاسخ‌گویی به سؤال‌ها از زمانبندی غیرقابل پس‌گیر استفاده کنید و تصمیم‌های خود را مطابق اطلاعاتی که بایستی در زمان تصمیم‌گیری داشته باشید، پایه‌گذاری کنید.

پردازه	زمان ورود	زمان انفجار
P_1	۰/۰	۸
P_2	۰/۴	۴
P_3	۱/۰	۱

الف- میانگین زمان گردش این پردازه‌ها با الگوریتم زمانبندی FCFS چقدر است؟

ب- میانگین زمان گردش این پردازه‌ها با الگوریتم زمانبندی SJF چقدر است؟

ج- انتظار می‌رود که الگوریتم SJF کارایی را بهبود ببخشد، اما توجه کنید که در لحظه‌ی ۰، پردازه‌ی P_1 را برای اجرا انتخاب می‌کنیم زیرا نمی‌دانیم که به زودی

دو پردازه‌ی کوتاه‌تر می‌رسند. میانگین زمان گردش را در صورتی محاسبه کنید که پردازنده به مدت ۱ واحد زمان بیکار بنشیند و سپس از زمانبندی SJF استفاده شود. به یاد داشته باشید که پردازه‌های P_1 و P_2 در مدت زمان بیکاری پردازنده، منتظر خواهند بود، لذا امکان دارد که زمان انتظار آنها افزایش پیدا کند. این الگوریتم به زمانبندی با دانش‌آتی معروف است.

۵-۴ مجموعه‌ی پردازه‌های زیر را که با طول زمان انفجار پردازنده‌هایشان به میلی‌ثانیه داده شده‌اند، در نظر بگیرید:

پردازه	زمان انفجار	الویت
P_1	۲	۲
P_2	۱	۱
P_3	۸	۴
P_4	۴	۲
P_5	۵	۳

فرض می‌شود که پردازه‌ها به‌ترتیب P_1، P_2، P_3، P_4، P_5، همگی در زمان ۰ رسیده باشند.

الف) چهار نمودار گانت ترسیم کنید که اجرای این پردازه‌ها را با استفاده از الگوریتم‌های زمان‌بندی FCFS، SJF، الویتی غیرقابل پس‌گیر (عدد الویت بزرگ‌تر به‌معنی الویت بالاتر است)، و RR (با کمیت = ۲) نمایش بدهند.

ب) زمان گردش هر پردازه برای هر یک از الگوریتم‌های زمان‌بندی قسمت (الف) چیست؟

ج) زمان انتظار هر پردازه برای هر یک از این الگوریتم‌های زمان‌بندی چیست؟

د) کدام یک از الگوریتم‌ها متوسط زمان انتظار کمینه را نتیجه می‌دهد (برای تمامی پردازه‌ها)؟

۵-۵ پردازه‌های زیر با استفاده از یک الگوریتم زمان‌بندی نوبت-چرخشی پس‌گیر زمان‌بندی می‌شوند.

پردازه	الویت	زمان انفجار	زمان ورود
P_1	۴۰	۲۰	۰
P_2	۳۰	۲۵	۲۵
P_3	۳۰	۲۵	۳۰
P_4	۳۵	۱۵	۶۰
P_5	۵	۱۰	۱۰۰

P_6	۱۰	۱۰	۱۰۵

به هر پردازه یک الویت عددی نسبت داده می‌شود که عدد بالاتر، نشان الویت نسبی بالاتر است. علاوه بر پردازه‌های لیست شده در بالا، سیستم یک **وظیفه‌ی بیکار** هم دارد (که هیچ‌گونه منبع پردازنده را مصرف نمی‌کند و به‌صورت P_{idle} مشخص می‌شود). الویت این پردازه ۰ است و زمانی زمان‌بندی می‌شود که سیستم هیچ پردازه‌ای برای اجرا نداشته باشد. طول کمیت زمانی برابر ۱۰ می‌باشد. اگر یک پردازه‌ای با الویت بالاتر پس‌گرفته شود، پردازه‌ی پس‌گرفته شده در انتهای صف گذاشته می‌شود.

الف) ترتیب زمان‌بندی پردازه‌ها را با استفاده از یک نمودار گانت نشان دهید.

ب) زمان گردش هر یک از پردازه‌ها چیست؟

ج) زمان انتظار هر یک از پردازه‌ها چیست؟

د) نرخ بهره‌وری پردازنده چیست؟

۵-۶ چه مزیتی در داشتن کمیت‌های زمانی با اندازه‌های متفاوت در سطوح متفاوت یک سیستم صف‌بندی چندسطحی وجود دارد؟

۵-۷ بسیاری از الگوریتم‌های زمانبندی پردازنده دارای پارامتر هستند. برای مثال، الگوریتم RR نیاز به پارامتری دارد که برش زمانی را مشخص نماید. صف‌های با بازخورد چندسطحی نیاز به پارامترهایی دارند که تعداد صف‌ها، الگوریتم‌های زمانبندی هر یک از صف‌ها، معیارهای مورد استفاده برای جابه‌جایی پردازه‌ها مابین صف‌ها و غیره، از آن جمله‌اند.

این الگوریتم‌ها به واقع مجموعه‌هایی از الگوریتم‌ها هستند (برای مثال، مجموعه‌ی الگوریتم‌های RR، برای تمامی برش‌های زمانی و سایر موارد مشابه). یک مجموعه الگوریتم می‌تواند شامل الگوریتم دیگر هم باشد (برای مثال، الگوریتم FCFS یک الگوریتم RR با کمیت زمانی بی‌نهایت است). چه رابطه‌ای (در صورت وجود) میان هر زوج از مجموعه الگوریتم‌های زیر برقرار است؟

الف- الویتی و SJF. ب- صف‌های با بازخورد چندسطحی و FCFS.

ج- الویتی و FCFS. د- RR و SJF.

۵-۸ فرض کنید که الگوریتم زمانبندی پردازه‌هایی را ترجیح می‌دهد که در گذشته‌ی اخیر، کمترین استفاده را از زمان پردازنده داشته‌اند. چرا این الگوریتم، برنامه‌های پاپیچ I/O را ترجیح خواهد داد و هنوز هم برنامه‌های پاپیچ پردازنده را به طور دایمی قطحی‌زده نمی‌کند؟

۵-۹ چه فرقی می‌توان میان زمانبندی PCS و SCS قایل شد.

۵-۱۰ زمانبند سابق یونیکس میان شمارگان الویت و الویت‌ها یک رابطه‌ی معکوسی اعمال می‌کند: شماره‌ی بالاتر، الویت پایین‌تر. زمانبند الویت‌های پردازه را در هر ثانیه یک بار با تابع زیر از نو محاسبه می‌کند:

الویت = پایه + (استفاده‌ی اخیر پردازنده / 2)

که در آن ۶۰ = پایه و *استفاده‌ی اخیر پردازنده*، مقداری است که نشان می‌دهد پردازه از زمان بار آخری که الویت آن از نو محاسبه شده است، چند مرتبه از پردازنده استفاده کرده است.

فرض کنید استفاده‌ی اخیر CPU برای پردازه‌ی P_1، ۴۰، P_2، ۱۸ و P_3، ۱۰ باشد. الویت‌های جدید این سه ئپردازه بعد از محاسبه‌ی دوباره‌ی الویت‌ها چقدر خواهند بود؟ آیا زمانبند سابق یونیکس، براساس این اطلاعات، الویت نسبی یک پردازه پاپیچ پردازنده را افزایش یا کاهش می‌دهد؟

تمرین‌ها

۵-۱۱ از میان این دو نوع برنامه:

الف) پاپیچ I/O ب) پاپیچ پردازنده

احتمالاً، کدام یک از آن‌ها برگردان‌های متن داوطلبانه‌ای خواهد داشت، و کدام یک احتمالاً برگردان‌های متن غیرداوطلبانه‌ای خواهد داشت؟ پاسخ خود را توضیح دهید.

۵-۱۲ توضیح دهید که چگونه هر یک از زوج معیارهای زمانبندی زیر در تنظیمات خاصی با هم در تضاد هستند.

الف) بهره‌وری پردازنده و زمان پاسخ

ب) متوسط زمان گردش و زمان انتظار بیشینه

ج) بهره‌وری دستگاه I/O و بهره‌وری پردازنده

۵-۱۳ یک تکنیک برای پیاده‌سازی **زمانبندی قرعه**[1] با دادن بلیت‌های قرعه برای پردازه‌ها کار می‌کند، این قرعه‌ها در تخصیص وقت پردازنده به‌کار می‌آیند. هر موقع که یک تصمیم زمانبندی صورت می‌گیرد، یک بلیت قرعه به تصادف انتخاب می‌شود و پردازه‌ای که آن بلیت را گرفته، پردازنده را در اختیار می‌گیرد. سیستم‌عامل BTV، زمانبندی قرعه را با ۵۰ بار قرعه‌کشی در هر ثانیه که برنده‌ی هر قرعه، ۲۰ میلی‌ثانیه از وقت پردازنده (۱ ثانیه = ۵۰ × ۲۰ میلی‌ثانیه) را به‌دست می‌آورد، پیاده‌سازی می‌کند. توضیح دهید که چگونه زمانبند BTV می‌تواند مطمئن شود که

1. Lottery scheduling

نخ‌های با الویت بالاتر نسبت به نخ‌های با الویت پایین‌تر از توجه بیشتری برخوردار می‌شوند.

۵-۱۴ اکثر الگوریتم‌های زمانبندی، یک **صف اجرا** نگه می‌دارند که شامل فهرست پردازه‌های آماده برای اجراست. در سیستم‌های چندهسته‌ای، دو انتخاب کلی وجود دارند: (۱) هر هسته‌ی پردازشی، صف اجرای خاص خود را داشته باشد، یا (۲) یک صف اجرای منفرد توسط تمامی هسته‌های پردازشی به اشتراک گذاشته شود. مزایا و معایب هر یک از این رویکردها کدامند؟

۵-۱۵ فرمول میانگین نمایی را که برای پیش‌بینی طول انفجار بعدی پردازنده استفاده می‌شود در نظر بگیرید. پیامدهای انتساب مقادیر زیر به پارامترهای مورد استفاده در الگوریتم چه خواهد بود؟

الف) $\alpha = ۰$ و $\tau_0 = ۱۰۰$ میلی‌ثانیه ب) $\alpha = ۰/۹۹$ و $\tau_0 = ۱۰$ میلی‌ثانیه

۵-۱۶ گونه‌ای از زمانبند نوبت-چرخشی، زمانبند **نوبت-چرخشی رگرسیو** است. این زمانبند به هر پردازه یک کمیت زمان و یک الویت اعطا می‌کند. مقدار اولیه‌ی کمیت زمانی ۵۰ میلی‌ثانیه است. با این وجود، هر بار که یک پردازه پردازنده را در اختیار می‌گیرد و کل کمیت زمانیش را استفاده می‌کند (برای I/O مسدود نمی‌شود)، ۱۰ میلی‌ثانیه به کمیت زمانیش افزوده شده و سطح الویت آن ارتقا می‌یابد. (کمیت زمانی یک پردازه می‌تواند حداکثر تا ۱۰۰ میلی‌ثانیه افزایش یابد.) هر زمان که پردازه قبل از این که کل کمیت زمانیش را استفاده کند، مسدود شود، کمیت زمانیش ۵ میلی‌ثانیه کاهش داده می‌شود، اما الویت آن یکسان باقی می‌ماند. زمانبند نوبت-چرخشی رگرسیو به چه نوع پردازه‌ای (پابند I/O یا پابند پردازنده) لطف بیشتری دارد؟ توضیح دهید.

۵-۱۷ مجموعه‌ی پردازه‌های زیر را با طول انفجار پردازنده برحسب میلی‌ثانیه در نظر بگیرید:

پردازه	زمان انفجار	الویت
P_1	۵	۴
P_2	۳	۱
P_3	۱	۲
P_4	۷	۲
P_5	۴	۳

فرض می‌شود که پردازه‌ها در لحظه‌ی ۰ به ترتیب P_1، P_2، P_3، P_4 و P_5 می‌رسند.

الف) چهار نمودار گانت ترسیم کنید که اجرای این پردازه‌ها را با استفاده از زمانبندی FCFS، SJF، الویتی غیرقابل پس‌گیر (شماره‌ی الویت بزرگ‌تر، نشانگر الویت بالاتر است) و RR (با کمیت = ۲) نشان بدهند.

ب) زمان گردش هر یک از پردازه‌ها با هر یک از الگوریتم‌های زمانبندی قسمت (الف) چقدر است؟

ج) زمان انتظار هر یک از پردازه‌ها با هر یک از الگوریتم‌های زمانبندی قسمت (الف) چقدر است؟

د) کدام یک از الگوریتم‌های زمانبندی، میانگین زمان انتظار کمینه (برای کل پردازه‌ها) را دارند؟

۵-۱۸ پردازه‌های زیر با استفاده از یک الگوریتم زمانبندی نوبت-چرخشی الویتی پس‌گرفتنی زمانبندی می‌شوند.

نخ	الویت	انفجار	ورود
P_1	۸	۱۵	۰
P_2	۳	۲۰	۰
P_3	۴	۲۰	۲۰
P_4	۴	۲۰	۲۵
P_5	۵	۵	۴۵
P_6	۵	۱۵	۵۵

برای هر پردازه یک الویت عددی که عدد بزرگ‌تر نشان الویت نسبی بالاتر است، انتخاب می‌شود. زمان‌بند، پردازه‌ی با بالاترین الویت را اجرا خواهد کرد. برای پردازه‌های با الویت یکسان، زمان‌بندی نوبت-چرخشی با طول کمیت زمانی ۱۰ واحد استفاده می‌شود. اگر پردازه‌ای توسط یک پردازه‌ی با الویت بالاتر پس گرفته شود، پردازه‌ی پس زده شده در انتهای صف قرار می‌گیرد.

الف) ترتیب زمانبندی پردازه‌ها را با استفاده از یک نمودار گانت نشان دهید.

ب) زمان گردش هر یک از پردازه‌ها چیست؟

ج) زمان انتظار هر یک از پردازه‌ها چیست؟

۵-۱۹ فرمان nice در لینوکس درکنار سیستم‌های یونیکس دیگر، برای انتخاب مقدار عالی یک پردازه استفاده می‌شود. توضیح دهید که چرا برخی سیستم‌ها ممکن است به هر کاربری اجازه بدهند که برای یک پردازه، یک مقدار عالی بزرگ‌تر یا مساوی ۰ را انتخاب کند، اما تنها به کاربر root (یا administrator) اجازه می‌دهند که مقادیر عالی کوچک‌تر از ۰ را انتخاب کند.

۵-۲۰ کدام یک از الگوریتم‌های زمانبندی زیر می‌تواند منجر به قحطی‌زدگی شود؟

الف) اولین-ورود، اولین-سرویس (FCFS)

ب) کوتاهترین-کار-اول (SJF)

ج) نوبت-چرخشی (RR)

د) الویتی

۵-۲۱ گونه‌ای از الگوریتم زمانبندی RR را در نظر بگیرید که واردهای صف آماده در آن، اشاره‌گرهایی به PCBها هستند.

الف) پیامد گذاشتن دو اشاره‌گر برای یک پردازه در صف آماده چیست؟

ب) دو مورد از مزایا و معایب مهم این طرح کدامند؟

ج) چگونه می‌توانید الگوریتم RR اصلی را تغییر بدهید تا بدون تکثیر اشاره‌گرها نیز بتوان به این هدف رسید؟

۵-۲۲ سیستمی را در نظر بگیرید که در آن ده وظیفه پاپیچ I/O و یک وظیفه پاپیچ پردازنده اجرا می‌شوند. فرض کنید وظیفه‌های پاپیچ I/O با هر یک میلی‌ثانیه از محاسبات پردازنده یک مرتبه عملیات I/O صادر می‌کنند و تکمیل هر عمل I/O، ۱۰ میلی‌ثانیه طول می‌کشد. همچنین فرض کنید که سربار برگردان متن ۰/۱ میلی‌ثانیه است و تمامی پردازه‌ها وظیفه‌هایی هستند که به مدت طولانی اجرا خواهند داشت. بهره‌وری پردازنده با زمانبند نوبت-چرخشی را در هر یک از حالت‌های زیر تشریح کنید؟

الف) با کمیت زمانی ۱ میلی‌ثانیه ب) با کمیت زمانی ۱۰ میلی‌ثانیه

۵-۲۳ سیستمی را در نظر بگیرید که زمانبندی صف چندسطحی را پیاده‌سازی می‌کند. کاربر کامپیوتر از چه ترفندی می‌تواند استفاده کند تا میزان زمان تخصیص پردازنده به پردازه‌ی خود را بیشینه سازد؟

۵-۲۴ الگوریتم زمانبندی الویتی پس‌گرفتنی را مطابق با تغییر پویای الویت‌ها در نظر بگیرید. شمارگان بزرگ‌تر الویت بر الویت بالاتر دلالت دارند. زمانی که پردازه‌ای منتظر پردازنده است (در صف آماده قرار دارد، اما اجرا ندارد)، الویت آن با نرخ α تغییر می‌یابد؛ زمانی که اجرا می‌شود، الویت آن با نرخ β تغییر می‌کند. تمامی پردازه‌ها در زمان ورود به صف آماده، الویت ۰ می‌گیرند. می‌توان با انتخاب پارامترهای α و β، الگوریتم‌های زمانبندی متفاوت زیادی بدست آورد.

الف) الگوریتم حاصل از $\beta > \alpha > 0$ چیست؟

ب) الگوریتم حاصل از $\alpha < \beta < 0$ چیست؟

۵-۲۵ توضیح دهید که چگونه الگوریتم‌های زمانبندی زیر به نفع یا به ضرر پردازه‌های کوتاه عمل می‌کنند:

الف) FCFS ب) RR ج) صف‌های با بازخورد چندسطحی

۵-۲۶ توضیح دهید که چرا یک صف آماده‌ی مشترک در یک محیط SMP ممکن است از مسائل کارایی رنج ببرد.

۵-۲۷ یک الگوریتم موازنه‌ی بار را در نظر بگیرید که تضمین می‌کند مستقل از الویت، هر صف تقریباً تعداد نخ یکسانی خواهد داشت. یک الگوریتم زمان‌بندی الویتی

چقدر می‌تواند کارآمد به این شرایط رسیدگی کند اگر یک صف اجرا تمامی نخ‌های الویت بالا و صف دوم، تمامی نخ‌های با الویت پایین را داشته باشد؟

۵-۲۸ فرض کنید که یک سیستم SMP دارای صف‌های خصوصی به‌ازای هر پردازنده است. موقعی که پردازه‌ی جدیدی ایجاد می‌شود، می‌تواند در همان صف پردازه‌ی والد و یا یک صف مجزایی قرار بگیرد.

الف) منافع گذاشتن پردازه‌ی جدید در همان صف والدش چیست؟

ب) منافع گذاشتن پردازه‌ی جدید در یک صف متفاوت چیست؟

۵-۲۹ فرض کنید که یک نخ برای I/O شبکه مسدود شده است و آمادگی دارد که دوباره اجرا شود. توضیح دهید که چرا یک الگوریتم زمان‌بندی NUMA بایستی نخ را دومرتبه بر روی همان CPU که قبلاً اجرا می‌شد زمان‌بندی نماید.

۵-۳۰ با استفاده از الگوریتم زمانبندی ویندوز، الویت عددی یک نخ را در هر یک از حالت‌های زیر بدست آورید.

الف) نخی از رده‌ی REALTIME_PRIORITY_CLASS با الویت نسبیNORMAL.

ب) نخی از رده‌ی ABOVE_NORMAL_PRIORITY_CLASS با الویت نسبی HIGHEST.

ج) نخی از رده‌ی BELOW_NORMAL_PRIORITY_CLASS با الویت نسبی ABOVE_NORMAL.

۵-۳۱ با این فرض که هیچ نخی به REALTIME_PRIORITY_CLASS تعلق نداشته و هیچکدام هم نمی‌توانند الویت TIME_CRITICAL را انتخاب کنند، چه ترکیبی از رده‌ی الویت و الویت، با بالاترین الویت نسبی ممکن در زمانبندی ویندوز متناظر است؟

۵-۳۲ الگوریتم زمانبندی سیستم‌عامل سولاریس را برای نخ‌های اشتراک‌زمانی در نظر بگیرید.

الف) کمیت زمانی (برحسب میلی‌ثانیه) یک نخ با الویت ۱۵ چقدر است؟ با الویت ۴۰ چقدر است؟

ب) فرض کنید که نخی با الویت ۵۰، کمیت زمانی خود را بدون انسداد به طور کامل استفاده کرده است. الویت جدیدی که زمانبند برای این نخ نسبت خواهد داد، چقدر است؟

ج) فرض کنید که نخی با الویت ۲۰ پیش از آنکه کمیت زمانیش به اتمام برسد برای I/O مسدود می‌شود. الویت جدیدی که زمانبند برای این نخ نسبت خواهد داد، چقدر است؟

۵-۳۳ فرض کنید که دو وظیفه‌ی A و B در یک سیستم لینوکس اجرا می‌شوند. مقادیر عالی A و B، به ترتیب ۵- و ۵+ می‌باشند. با استفاده از زمانبند CFS به عنوان یک راهنما، توضیح دهید که چگونه مقادیر متغیر vruntime مربوط به این دو پردازه

در هر یک از شرایط زیر تغییر می‌کند:

- هر دوی پردازه‌ها پابند پردازنده هستند.
- A پابند I/O و B پابند پردازنده است.
- A پابند پردازنده و B پابند I/O است.

۵-۳۴ شرایط خاصی را فراهم کنید که نشان می‌دهد زمان‌بندی نرخ-یکنواخت در برآوردن ضرب‌الاجل‌های پردازه‌ی بلادرنگ نسبت به زمان‌بندی قدیمی‌ترین-ضرب‌الاجل-اول کوتاهی می‌کند.

۵-۳۵ دو پردازه‌ی P_1 و P_2 را که $p_1 = 50$، $t_1 = 25$، $p_2 = 75$ و $t_2 = 30$ در نظر بگیرید.

الف) آیا امکان زمانبندی این دو پردازه با استفاده از زمانبندی نرخ-یکنواخت وجود دارد؟ پاسخ خود را با استفاده از یک نمودار گانت، نظیر آنهایی که در شکل ۵-۲۱ تا شکل ۵-۲۴ نشان داده شدند، نمایش دهید.

ب) زمانبندی این دو پردازه را با استفاده از زمانبندی قدیمی‌ترین-ضرب‌الاجل-اول (EDF) نشان دهید.

۵-۳۶ توضیح دهید که چرا بایستی زمان‌های وقفه و دیرکرد اعزام در یک سیستم بلادرنگ سخت کراندار باشند.

۵-۳۷ مزایای استفاده از چندپردازشی ناهمگن را در یک سیستم موبایل توضیح دهید.

پروژه‌های برنامه‌نویسی

الگوریتم‌های زمان‌بندی

این پروژه شامل پیاده‌سازی چندین الگوریتم متفاوت زمان‌بندی پردازه است. یک مجموعه‌ای از وظایف از پیش تعریف شده‌ای به زمان‌بند گمارده می‌شود که وظایف را بر اساس الگوریتم زمان‌بندی انتخابی زمان‌بندی خواهد کرد. به هر وظیفه یک الویت و یک انفجار پردازنده نسبت داده می‌شود. الگوریتم‌های زمان‌بندی زیر پیاده‌سازی خواهند شد:

- اولین-ورود، اولین-سرویس (FCFS) که وظایف را به‌ترتیبی که پردازنده را درخواست می‌دهند زمان‌بندی می‌نماید.
- کوتاه‌ترین-کار-اول (SJF) که وظایف را به‌ترتیب طول انفجار پردازنده‌ی بعدی وظایف زمان‌بندی می‌نماید.
- زمان‌بندی الویتی که وظایف را بر اساس الویت زمان‌بندی می‌نماید.
- زمان‌بندی نوبت-چرخشی (RR) که در آن، هر وظیفه برای یک کمیت زمانی (یا برای باقیمانده‌ی انفجار پردازنده‌اش) اجرا می‌شود.
- الویتی با نوبت-چرخشی که وظایف را به‌ترتیب الویت زمان‌بندی کرده و برای

وظایف با الویت برابر از زمان‌بندی نوبت-چرخشی استفاده می‌کند.
الویت‌ها در بازه‌ی ۱ تا ۱۰ هستند که مقدار عددی بالاتر، الویت نسبی بالاتر را نشان می‌دهد. در الگوریتم نوبت-چرخشی، طول کمیت زمانی برابر ۱۰ میلی‌ثانیه می‌باشد.

۱—پیاده‌سازی

پیاده‌سازی این پروژه می‌تواند به‌زبان C یا جاوا تکمیل شود، و فایل برنامه‌های پشتیبانی هر دو زبان در کد اصلی دانلود شده برای این کتاب تدارک دیده شده‌اند. این فایل‌های پشتیبان، زمان‌بندی وظایف را خوانده، وظایف را در یک لیست وارد کرده و زمان‌بند را احضار می‌کنند.
زمان‌بندی وظایف به‌شکل [**نام وظیفه**][**الویت**][**انفجار پردازنده**]، با قالب نمونه‌ی زیر می‌باشد:

```
T1, 4, 20
T2, 2, 25
T3, 3, 25
T4, 3, 15
T5, 10, 10
```

لذا، وظیفه‌ی T1 دارای الویت ۴ و انفجار پردازنده‌ی ۲۰ میلی‌ثانیه می‌باشد، و الی‌آخر. فرض می‌شود که تمامی وظایف همزمان می‌رسند، لذا الگوریتم‌های زمان‌بندی شما مجبور نیستند که از پس‌گیری پردازه‌های با الویت بالاتر از پردازه‌های با الویت پایین‌تر پشتیبانی نمایند. به‌علاوه، وظایف مجبور نیستند که در یک صف یا لیست با هر ترتیب خاصی گذاشته شوند.

چند راهبرد متفاوت برای سازمان‌دهی لیست وظایف وجود دارد، همان‌گونه که در ابتدا در بخش ۵-۱-۲ مطرح شد. یک رویکرد این است که تمامی وظایف در یک لیست بدون ترتیب منفردی گذاشته شوند که راهبرد انتخاب وظیفه به الگوریتم زمان‌بندی بستگی دارد. برای مثال، زمان‌بندی SJF لیست را جستجو خواهد کرد تا وظیفه‌ای را پیدا کند که کوتاه‌ترین انفجار پردازنده‌ی بعدی را دارد. به‌روشی دیگر، لیست می‌تواند مطابق معیارهای زمان‌بندی (یعنی، بر اساس الویت) مرتب شود. یک راهبرد دیگر مشمول یک صف مجزایی برای هر الویت یکتاست، چنان‌چه در شکل ۵-۷ نشان داده می‌شود. این رویکردها به‌اختصار در بخش ۵-۳-۶ بحث می‌شوند. همین‌طور بایستی تأکید دارد که ما از تعابیر **لیست** و **صف** تا حدودی به جای همدیگر استفاده می‌کنیم. با این وجود، یک صف کارکرد FIFO بسیار خاصی دارد، در حالی که یک لیست، این مدل نیازمندی‌های مقید درج و حذف را ندارد. شما احتمالاً کارکرد یک لیست عمومی را موقع تکمیل این پروژه مناسب‌تر خواهید یافت.

۲—جزئیات پیاده‌سازی به‌زبان C

فایل driver.c زمان‌بندی وظایف را می‌خواند، هر وظیفه را در یک لیست پیوندی درج می‌کند، و زمان‌بند پردازه را با فراخوانی تابع ()schedule احضار می‌نماید. تابع ()schedule هر وظیفه را مطابق الگوریتم زمان‌بندی مشخص شده اجرا می‌کند. وظایف انتخابی برای اجرا بر روی پردازنده توسط تابع ()pickNextTask تعیین شده و با احضار تابع ()run که در فایل CPU.c تعریف شده است اجرا می‌شوند. برای تعیین یک الگوریتم زمان‌بندی خاص از یک Makefile

استفاده می‌شود که توسط driver احضار خواهد شد. برای مثال، به‌منظور ساخت زمان‌بند FCFS فرمان زیر را وارد می‌کنیم

```
make fcfs
```

و زمان‌بند را (با استفاده از زمان‌بندی وظایف، schedule.txt) به‌صورت زیر اجرا خواهیم کرد:

```
./fcfs schedule.txt
```

برای جزئیات بیشتر به فایل README در کد اصلی دانلود شده مراجعه کنید. قبل از ادامه کار مطمئن شوید که با کد اصلی فراهم شده، همین‌طور Makefile آشنایی دارید.

۳—جزئیات پیاده‌سازی به‌زبان جاوا

فایل Driver.java زمان‌بندی وظایف را خوانده، هر وظیفه را در یک ArrayList جاوا درج کرده، و زمان‌بند پردازه را با فراخوانی متد ()schedule احضار می‌نماید. رابط زیر، الگوریتم زمان‌بندی عمومی را مشخص می‌کند که پنج الگوریتم متفاوت زمان‌بندی پیادسازی خواهند کرد:

```
public interface Algorithm
{
        // Implementation of scheduling algorithm
        public void schedule();
        // Selects the next task to be scheduled
        public Task pickNetTask();
}
```

متد ()schedule وظیفه‌ی بعدی را که قرار است بر روی پردازنده اجرا شود با احضار متد ()pickNextTask بدست آورده و سپس، این وظیفه را با فراخوانی متد ایستای ()run در کلاس CPU.java اجرا می‌کند.

برنامه به‌صورت زیر اجرا می‌شود:

```
java Driver fcfs schedule.txt
```

برای جزئیات بیشتر به فایل README در کد اصلی دانلود شده مراجعه کنید. قبل از ادامه‌ی کار، مطمئن شوید که با تمامی فایل‌های کدهای اصلی جاوا که در دانلود کد اصلی فراهم شده آشنایی دارید.

۴—چالش‌های بیشتر

دو چالش دیگر برای این پروژه به‌صورت زیر ارائه می‌شود:

۱- به هر وظیفه‌ای که برای زمان‌بند مهیا می‌شود یک شناسه‌ی وظیفه‌ی یکتا (tid) نسبت داده می‌شود. اگر یک زمان‌بند در یک محیط SMP اجرا داشته باشد که هر پردازنده به‌صورت جداگانه زمان‌بند خاص خود را اجرا می‌کند، یک شرایط مسابقه بر روی متغیری که برای انتساب شناسه‌های وظایف استفاده می‌شود امکان‌پذیر است. این

شرط مسابقه را با استفاده از یک مقدار صحیح اتمیک درست کنید.

در سیستم‌های لینوکس و macOS، تابع __sync_fetch_and_add() می‌تواند به‌صورت خودکار یک مقدار عددی صحیح را افزایش دهد. به‌عنوان یک مثال، کد نمونه‌ی زیر، مقدار value را به‌صورت خودکار ۱ واحد افزایش می‌دهد:

```
int value = 0;
__sync_fetch_and_add(&value,1);
```

برای جزئیات مربوط به چگونگی استفاده از کلاس AtomicInteger در برنامه‌های جاوا به API جاوا مراجعه کنید.

۲- متوسط زمان گردش، زمان انتظار، و زمان پاسخ را برای هر یک از الگوریتم‌های زمان‌بندی محاسبه کنید.